

[← Go Back](#)

Hardware

Nano 33 BLE Rev2

Tutorials

[Nano 33 BLE Rev2 User Manual](#)

Get Started With Machine Learning

[Accessing Accelerometer Data on Nano 33 BLE Rev2](#)[Accessing Gyroscope Data on Nano 33 BLE Rev2](#)[Accessing Magnetometer Data on Nano 33 BLE Rev2](#)[Home](#) / [Hardware](#) / [Nano 33 BLE Rev2](#) / [Get Started With Machine Learning](#)

ON THIS PAGE

Get Started With Machine Learning

Learn how to train and use machine learning models with the Arduino Nano 33 BLE Rev2

Author · Sandeep Mistry, Dominic Pajak, and Hannes Siebeneicher

Last revision · 03/11/2024

This post was originally published by Sandeep Mistry and Dominic Pajak on the [TensorFlow blog](#) and uses the Arduino [Nano 33 BLE Sense Rev2](#).

i This revision of the tutorial is adapted to the [Arduino Nano 33 BLE Rev2](#) which comes without a microphone and therefore leaves out the speech recognition.

Introduction

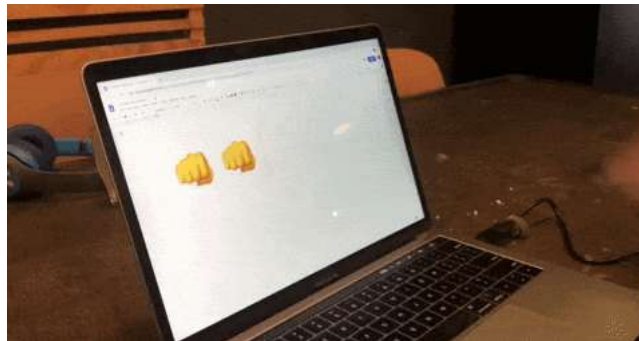
Important notice! The [TensorFlow Lite Micro Library](#) is no longer

i available in the Arduino Library Manager. This library will need to be manually downloaded and included in your IDE.

In this article, we'll show you how to install and run several [TensorFlow Lite Micro](#) examples.

We'll talk about how you can train your custom gesture recognition model for Arduino using TensorFlow in Colab. This material is based on a practical workshop held by Sandeep Mistry and Don Coleman, an updated version of which is [online here](#).

If you have previous experience with Arduino, you may be able to get these tutorials working within a couple of hours. If you're entirely new to microcontrollers, it may take a bit longer.



We're excited to share some examples and tutorials, and to see what you will build from here. Let's get started!

Introduction

[Goals](#)[Hardware & Software N](#)[Microcontrollers and Ti](#)[TensorFlow Lite for Microcontrollers Examp](#)[How To Run The Examp Using the Arduino IDE](#)[Training a TensorFlow L Micro Model For Arduin](#)[IDE Setup](#)[Streaming Sensor Data the Arduino Board](#)[Visualizing Live Sensor I Log From the Arduino B](#)[Capturing Gesture Trair Data](#)[Training in TensorFlow](#)[Classifying IMU Data](#)[Conclusion](#)

Machine Learning on Arduino with TensorFlow Lite Micro



Note: The following projects are based on [TensorFlow Lite for Microcontrollers](#).

Goals

Learn the fundamentals of TinyML implementation and training.

Use the [BMI270_BMM150](#) and [Arduino_TensorFlowLite](#) libraries

Hardware & Software Needed

An [Arduino Nano 33 BLE Rev2](#) board

A Micro USB cable to connect the Arduino board to your desktop machine

To program your board, you can use the [Arduino Cloud Editor](#) or install the [Arduino IDE](#). We'll give you more details on how to set these up in the following sections

The Arduino Nano 33 BLE Rev2 has a 9-axis IMU (accelerometer, gyroscope, magnetometer) onboard, which gives it some potential for cool TinyML applications.

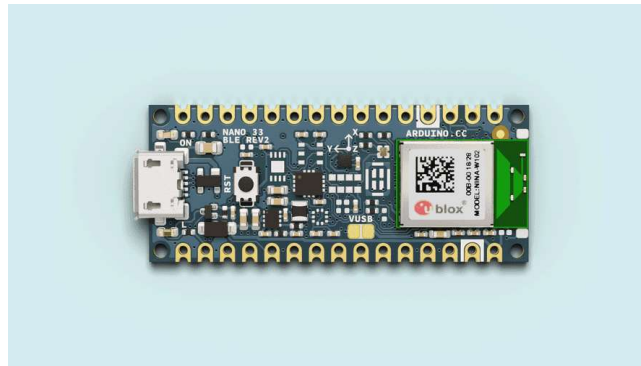
Unlike classic Arduino UNO, the board combines a microcontroller with onboard sensors which means you can address many use cases without additional hardware or wiring. The board is also small enough to be used in end applications like wearables. As the name suggests it has Bluetooth® Low Energy connectivity so you can send data (or inference results) to a laptop, mobile app or other Bluetooth® Low Energy boards and peripherals.

Tip: Sensors on a USB stick – Connecting the BLE board over USB is an easy way to capture data and add multiple sensors to single board computers without the need for additional wiring or hardware – a nice addition to a Raspberry Pi, for example.

Microcontrollers and TinyML

Microcontrollers, such as those used on Arduino boards, are low-cost, single-chip, self-contained computer systems. They're the invisible computers embedded inside billions of everyday gadgets like wearables, drones, 3D printers, toys, rice cookers, smart plugs, e-scooters, and washing machines. The trend to connect these devices is part of what is referred to as the Internet of Things.

Arduino is an open-source platform and community focused on making microcontroller application development accessible to **everyone**. The **board** we're using here has an Arm® Cortex®-M4 microcontroller running at 64 MHz with 1 MB Flash memory and 256 kB of RAM. This is tiny in comparison to cloud, PC, or mobile but reasonable by microcontroller standards.



The Arduino Nano 33 BLE Rev2 board comes in a tiny form factor

There are practical reasons you might want to squeeze ML on microcontrollers, including:

- Function – wanting a smart device to act quickly and locally (independent of the Internet).

- Cost – accomplishing this with simple, lower-cost hardware.

- Privacy – not wanting to share all sensor data externally.

- Efficiency – smaller device form factor, energy-harvesting or longer battery life.

There's a final goal that we're building towards that is very important:

- Machine learning can make microcontrollers accessible to developers who don't have a background in embedded development

On the machine learning side, there are techniques you can use to fit neural network models into memory-constrained devices like microcontrollers. One of the key steps is the **quantization of the weights** from floating point to 8-bit integers. This also has the effect of making inference quicker to calculate and more applicable to lower clock-rate devices.

TinyML is an emerging field and there is still work to do – but what's exciting is there's a vast unexplored application space out there. Billions of microcontrollers combined with all sorts of sensors in all sorts of places can lead to some seriously creative and valuable TinyML applications in the future.

TensorFlow Lite for Microcontrollers

Examples

The inference examples for TensorFlow Lite for Microcontrollers are now packaged and available through the Arduino Library Manager making it possible to include and run them on Arduino in a few clicks. In this section, we'll show you how to run them. The examples are:

magic_wand – gesture recognition using the onboard IMU
person_detection – person detection using an external ArduCam® camera

For more background on the examples, you can take a look at the source in the [TensorFlow repository](#). The models in these examples were previously trained. The tutorials below show you how to deploy and run them on an Arduino board. In the next section, we'll discuss training.

How To Run The Examples Using the Arduino IDE

Follow the instructions in the next section setting up the Arduino IDE.

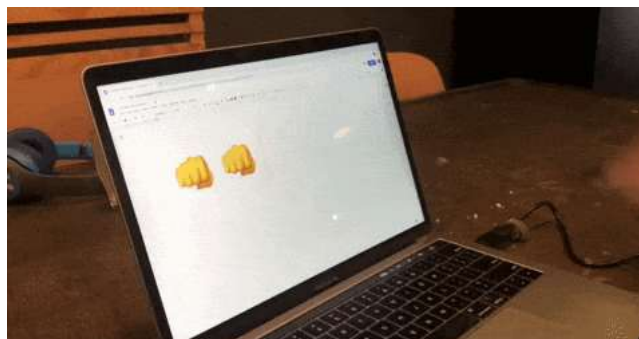
In the Arduino IDE, you will see the examples available via the **File > Examples > Arduino_TensorFlowLite** menu.

Select an example and the sketch will open. To compile, upload and run the examples on the board click the arrow icon:



For advanced users who prefer a command line, there is also the `arduino-cli`.

Training a TensorFlow Lite Micro Model For Arduino



import it into TensorFlow to train a model, and deploy the resulting classifier onto the board.

The idea for this tutorial was based on Charlie Gerard's awesome [Play Street Fighter with body movements using Arduino and Tensorflow.js](#). In Charlie's example, the board is streaming all sensor data from the Arduino board to another machine which performs the gesture classification in Tensorflow.js. We take this further and "TinyML-ify" it by performing gesture classification on the Arduino board itself. This is made easier in our case as the Arduino Nano 33 BLE Rev2 board we're using has a more powerful Arm® Cortex®-M4 processor and an onboard IMU.

We've adapted the tutorial below, so no additional hardware is needed – the sampling starts once movement is detected. The original version of the tutorial adds a breadboard and a hardware button to press to trigger sampling. If you want to get into a little hardware, you can follow that version [instead](#).

IDE Setup

1. First, let's make sure you have the [Nano 33 BLE board package](#) installed.
2. Also, let's make sure we have all the libraries we need installed.

 Important notice! The [TensorFlow Lite Micro Library](#) is no longer available in the Arduino Library Manager. This library will need to be manually downloaded and included in your IDE.

There are more detailed [Getting Started](#) and [Troubleshooting](#) guides on the Arduino site if you need help.

Streaming Sensor Data From the Arduino Board

First, we need to capture some training data. You can capture sensor data logs from the Arduino board over the same USB cable you use to program the board with your laptop or PC.

Arduino boards run small applications (also called sketches) which are compiled from .ino format Arduino source code, and programmed onto the board using the IDE or Cloud Editor.

With the sketch we are creating we will do the following:

- Monitor the board's accelerometer and gyroscope

- Trigger a sample window on detecting significant linear acceleration of the board

- Sample for one second at 119 Hz, outputting CSV format data over USB

- Loop back and monitor for the next gesture

The sensors we choose to read from the board, the sample rate, the trigger threshold, and whether we stream data output as CSV, JSON, binary or some other format are all customizable in the sketch running on the Arduino board. There is also scope to perform signal preprocessing and filtering on the device before the data is output to the log – this we can cover in another blog. For now, you can just upload the sketch and get a sampling.

The complete sketch can be found below:

```
1  /*
2   IMU Capture
3   This example uses the on-board IMU to start reading
4   data from on-board IMU and prints it to the Serial
5   when the significant motion is detected.
6   You can also use the Serial Plotter to graph the d
7   The circuit:
8   - Arduino Nano 33 BLE Rev2 board.
9   Created by Don Coleman, Sandeep Mistry
10  Modified by Dominic Pajak, Sandeep Mistry, Hannes
11  This example code is in the public domain.
12  */
13
14  #include <Arduino_BMI270_BMM150.h>
15
16  const float accelerationThreshold = 2.5; // threshol
17  const int numSamples = 119;
18
19  int samplesRead = numSamples;
20
21  void setup() {
22    Serial.begin(9600);
23    while (!Serial);
24
25    if (!IMU.begin()) {
26      Serial.println("Failed to initialize IMU!");
27      while (1);
28    }
29  }
```

Visualizing Live Sensor Data Log From the Arduino Board

With that done we can now visualize the data coming off the board. We're not capturing data yet this is just to give you a feel for how the sensor data capture is triggered and how long a sample window is. This will help when it comes to collecting training samples.

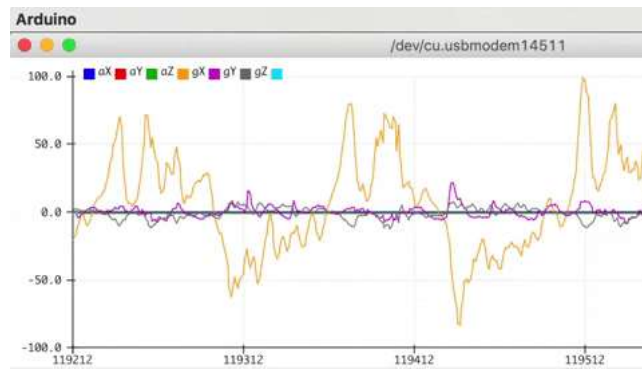
In the Arduino IDE, open the Serial Plotter **Tools > Serial Plotter**

If you get an error that the board is not available, reselect the port:

Tools > Port > portname (Arduino Nano 33 BLE)

Pick up the board and practice your punch and flex gestures

You'll see it only sample for a one-second window, then wait for the next gesture



When you're done be sure to close the Serial Plotter window – this is important as the next step won't work otherwise.

Capturing Gesture Training Data

To capture data as a CSV log to upload to TensorFlow, you can use **Arduino IDE > Tools > Serial Monitor** to view the data and export it to your desktop machine:

Reset the board by pressing the small white button on the top

Pick up the board in one hand (picking it up later will trigger sampling)

In the Arduino IDE, open the Serial Monitor Tools > Serial Monitor

If you get an error that the board is not available, reselect the port:

Tools > Port > portname (Arduino Nano 33 BLE)

Make a punch gesture with the board in your hand (Be careful whilst doing this!)

Make the outward punch quickly enough to trigger the capture

Return to a neutral position slowly so as not to trigger the capture again

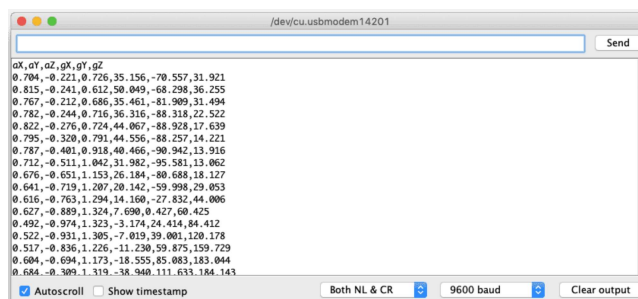
Repeat the gesture capture step 10 or more times to gather more data

Copy and paste the data from the Serial Console to a new text file called punch.csv

Clear the console window output and repeat all the steps above, this time with a flex gesture in a file called flex.csv

Make the inward flex fast enough to trigger capture returning slowly each time

Note: the first line of your two csv files should contain the fields aX,aY,aZ,gX,gY,gZ.

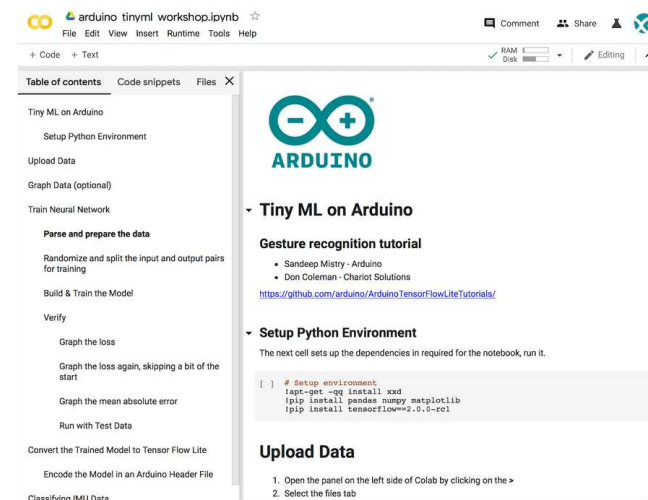


Linux tip: If you prefer you can redirect the sensor log output from the Arduino board straight to .csv file on the command line. With the Serial Plotter / Serial Monitor windows close use:

```
1 $ cat /dev/cu.usbmodem[nnnnn] > sensorlog.csv
```

Training in TensorFlow

We're going to use [Google Colab](#) to train our machine-learning model using the data we collected from the Arduino board in the previous section. Colab provides a Jupyter notebook that allows us to run our TensorFlow training in a web browser.



Arduino gesture recognition training colab.

Set up Python environment

Upload the punch.csv and flex.csv data

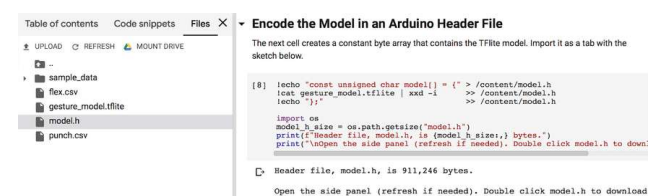
Parse and prepare the data

Build and train the model

Convert the trained model to TensorFlow Lite

Encode the model in an Arduino header file

The final step of the colab is to generate the model.h file to download and include in our Arduino IDE gesture classifier project in the next section:



Gesture classifier

Let's open the notebook in Colab and run through the steps in the cells – [arduino_tinyml_workshop.ipynb](#)

Classifying IMU Data

Next we will use model.h file we just trained and downloaded from Colab in the previous section of our Arduino IDE project:

We will be starting a new sketch, you will find the complete code below:

```

1  /*
2   IMU Classifier
3   This example uses the on-board IMU to start reading
4   data from on-board IMU, once enough samples are read, it then uses a
5   TensorFlow Lite (Micro) model to try to classify the movement as a known gesture.
6   Note: The direct use of C/C++ pointers, namespaces, and dynamic memory is generally
7   discouraged in Arduino examples, and in the future the TensorFlow Lite library
8   might change to make the sketch simpler.
9   The circuit:
10  - Arduino Nano 33 BLE Rev2 board.
11  Created by Don Coleman, Sandeep Mistry
12  Modified by Dominic Pajak, Sandeep Mistry
13  This example code is in the public domain.
14  */
15
16  #include "Arduino_BMI270_BMM150.h"
17
18  #include <TensorFlowLite.h>
19  #include <tensorflow/lite/micro/all_ops_resolver.h>
20  #include <tensorflow/lite/micro/micro_interpreter.h>
21  #include <tensorflow/lite/schema/schema_generated.h>
22
23  #include "model.h"
24
25  const float accelerationThreshold = 2.5; // threshold for acceleration
26  const int numSamples = 119;
27
28  int samplesRead = numSamples;

```

Create a new tab in the IDE. When asked name it model.h

Open the model.h tab and paste in the version you downloaded from Colab



Opening a new tab in your sketch

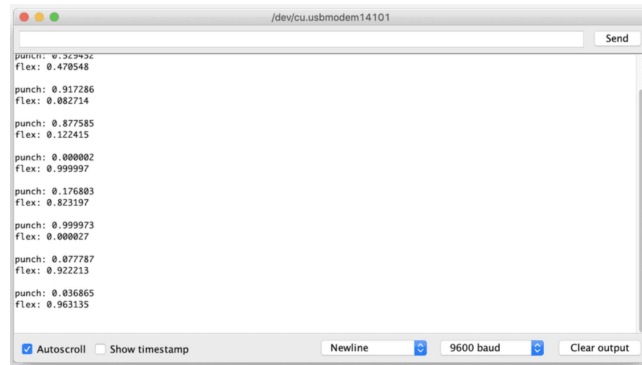
Upload the sketch: Sketch > Upload

Open the Serial Monitor: Tools > Serial Monitor

Perform some gestures

The confidence of each gesture will be printed to the Serial Monitor

Congratulations you've just trained your first ML application for Arduino!



Guessing the gesture with a confidence score

For added fun, the [Emoji_Button.ino](#) example shows how to create a USB keyboard that prints an emoji character in Linux and macOS. Try combining the [Emoji_Button.ino](#) example with the [IMU_Classifier.ino](#) sketch to create a gesture-controlled emoji keyboard.

Conclusion

It's an exciting time with a lot to learn and explore in TinyML. We hope this blog has given you some idea of the potential and a starting point to start applying it in your own projects. Be sure to let us know what you build and [share it](#) with the Arduino community.

For a comprehensive background on TinyML and the example applications in this article, we recommend Pete Warden and Daniel Situnayake's O'Reilly book "[TinyML: Machine Learning with TensorFlow on Arduino and Ultra-Low Power Microcontrollers](#)."

Suggest changes

The content on [docs.arduino.cc](#) is facilitated through a public [GitHub repository](#). If you see anything wrong, you can edit this page [here](#).

Need support?

[Help Center](#)
[Ask the Arduino Forum](#)
[Discover Arduino Discord](#)

License

The Arduino documentation is licensed under the [Creative Commons Attribution-Share Alike 4.0](#) license.

Was this article helpful?



